

Distribuirani sistemi

Konzistentnost sa stanovišta klijenta (client-centric)

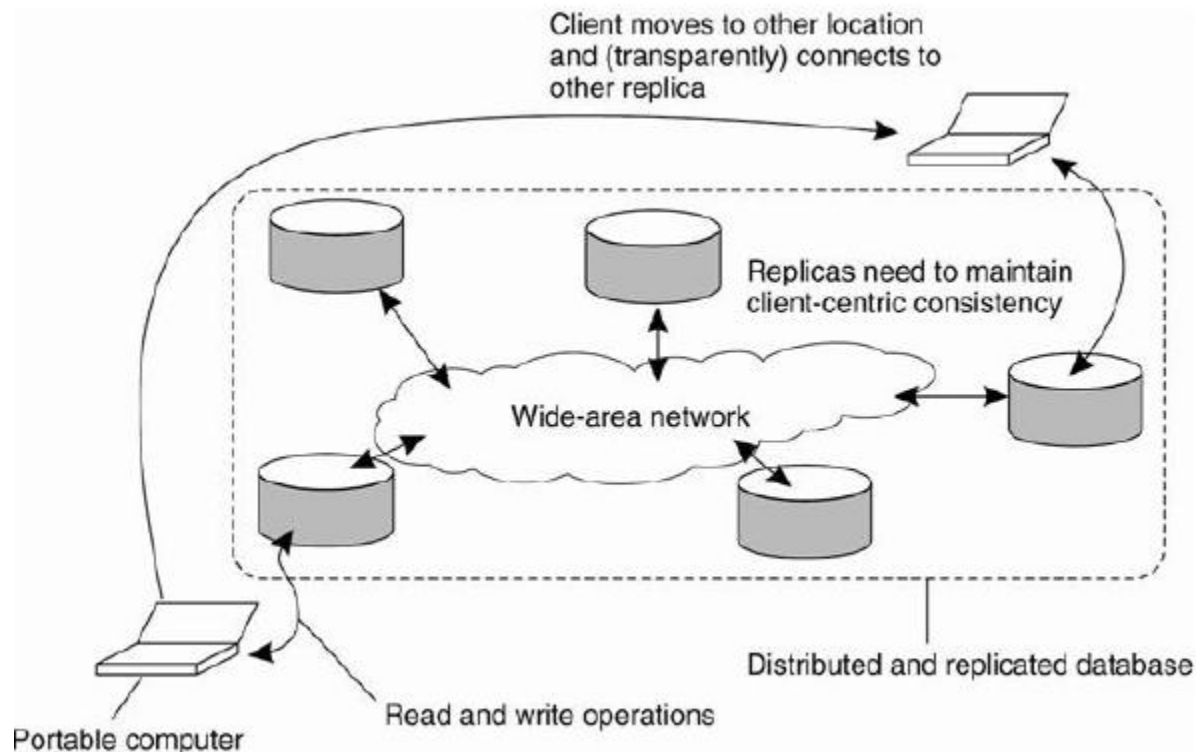
Pozicioniranje replika

Protokoli konzistencije

konzistentnost sa stanovišta klijenta

* Client-centric consistency

- ovde je naglasak na održavanju konzistentnog pogleda na skladište podataka sa stanovište pojedinačnog klijentskog procesa
 - klijent je mobilan i može sa različitih lokacijama čitati/modifikovati kopije skladišta podataka.
- postavlja se pitanje koje promene klijent može videti kada promeni lokaciju



Client-centric modeli konzistencije

* postoje četiri različita modela konzistencije posmatrano u odnosu na klijenta:

- monotona čitanja (monotonic reads)
- monotoni upisi (monotonic writes)
- čitaj svoje upise (read your writes)
- upisi nakon čitanja (writes follow reads)

* Označavanje

- $x_i[t]$ označava verziju podatka x u lokalnoj kopiji L_i u trenutku t
- $WS\ x_i[t]$ označava skup write operacija na lokalnoj kopiji L_i koje su dovele do verzije x_i promenljive x (u trenutku t)
- ako se set operacija $WS\ x_i[t_1]$ obavio i na lokalnoj kopiji L_j a zatim i novo ažuriranje na lokaciji L_j to se označava sa $WS(x_i[t_1], x_j[t_2])$
 - ako je vremenski redosled očigledan, oznaka vremena se može izostaviti

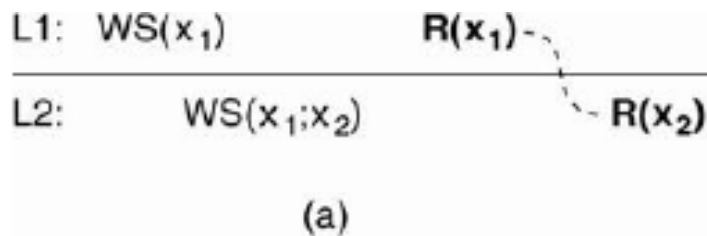
monotona čitanja (monotonic reads)

* Ako proces pročita vrednost podatka x na jednoj lokaciji, svako novo čitanje tog podatka će vratiti istu ili noviju vrednost bez obzira na kojoj lokaciji se obavlja čitanje.

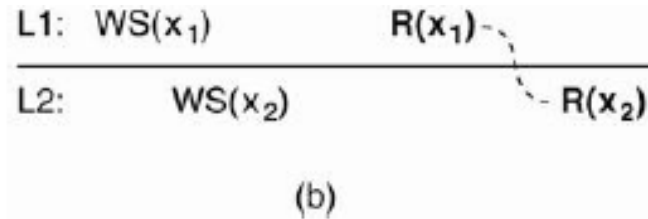
- Primer: čitanje email-a dok je klijent u pokretu

➤ svaki put kada se klijent poveže na drugi e-mail server klijent vidi sve mailove koje je video prethodnog puta, a može i novije

- a) Proces P prvo čita x na lokaciji $L1$ i vidi vrednost x_1
- ova vrednost je rezultat operacije $WS(x_1)$ koja je obavljena na lokaciji $L1$
 - Kasnije P čita x sa lokacije $L2$, što je označeno sa $R(x_2)$
 - da bi se garantovala konzistentost, skup operacija $WS(x_1)$ mora da budu propagiran i na lokaciju $L2$ pre drugog čitanja:
 - na lokaciji $L2$ je nakon ažuriranja $WS(x_1)$ obavljeno ažuriranje $WS(x_2)$ što je označeno sa $WS(x_1, x_2)$



monotona čitanja (monotonic reads)



* situacija u kojoj konzistencija tipa monotonic-read nije garantovana

- nakon što je proces P pročitao $x=x_1$ na lokaciji L1, kasnije čita $x=x_2$ sa lokacije L2
 - na lokaciji L2 su obavljene samo operacije $WS(x_2)$
 - nema garancija da ovaj skup $WS(x_2)$ sadrži sve operacije koje su bile obavljene u skupu $WS(x_1)$

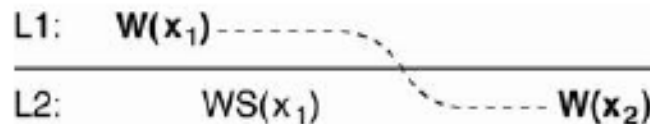
Monotoni upisi (Monotonic Writes)

* kod ovog modela konzistencije važe sledeći uslovi:

- Ako proces P obavlja modifikaciju (write) podatka x na lokaciji L1, proces P može obaviti modifikaciju podatka x na lokaciji L2 tek nakon što se na toj lokaciji obavi ažuriranje sa lokacije L1.

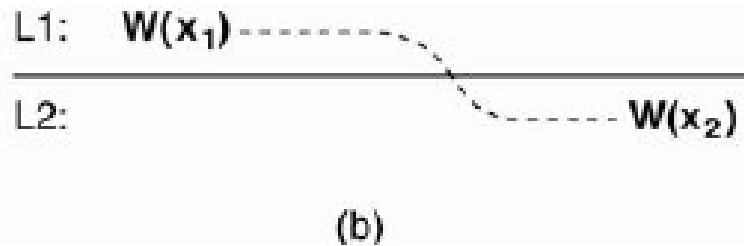
* Primer

- proces P obavlja write operaciju na lokaciji L1, $X(x_1)$
- kasnije proces P obavlja drugu write operaciju nad x na lokaciji L2, $W(x_2)$
- da bi se ispoštovala konzistencija "monotoni upisi" prethodna write operacija sa L1 mora biti prosleđena na lokaciju L2
 - to je označeno sa $WS(x_1)$ na lokaciji L2 pre obavljanja $W(x_2)$



(a)

Monotoni upisi (Monotonic Writes)

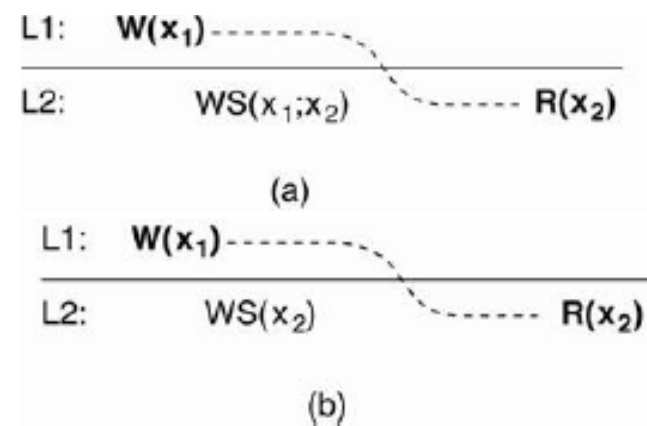


* situacija u kojoj konzistencija monotonic-write nije zagarantovana

- nedostaje propagiranje ažuriranja $W(x_1)$ sa lokacije L1 na lokaciju L2
- nema garancija da kopija podatka x na kojoj je obavljena druga write operacija ima istu ili noviju vrednost od $W(x_1)$ sa lokacije L1

Čitaj svoje upise (Read-your-writes)

- * model konzistencije koji je veoma blizak modelu monotonog čitanja
- * skladište podataka obezbeđuje read-your-writes konzistenciju ako je zadovoljeno sledeće:
 - efekat write operacije koju obavlja proces P nad podatkom x će uvek biti viđen kasnijim čitanjem podataka x od strane istog procesa



a) Process P obavlja write operaciju $W(x_1)$ na lokaciji L1, a kasnije obavlja čitanje sa lokacije L2

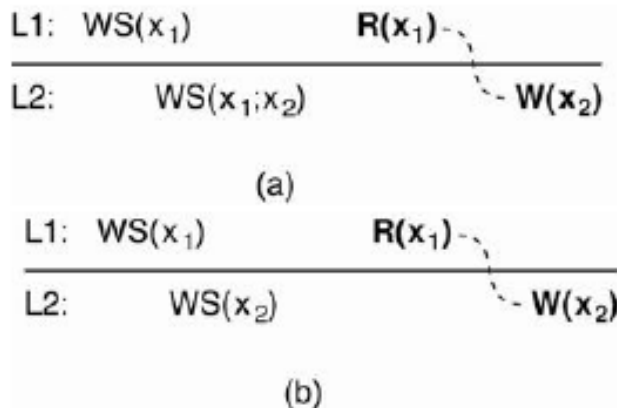
- Read-your-writes konsistencija garantuje da će efekat write operacije $W(x_1)$ biti vidljiv od strane kasnijih čitanja
 - to je označeno sa $WS(x_1; x_2)$, koje ukazuje da je $W(x_1)$ obuhvaćeno sa $WS(x_2)$

b) $W(x_1)$ nije obuhvaćeno sa $WS(x_2)$, što znači da efekat prethodne write operacije procesa P nije propagiran na lokaciju L2

upis sledi čitanje (Writes Follow Reads)

* skladište podataka obezbeđuje writes-follow-reads konzistenciju ako važi sledeće:

- write operacija koju je obavio proces P nad podatkom x nakon prethodnog čitanja tog podatka se obavlja nad istom ili novijom vrednošću podatka x



- proces čita x na lokalnoj kopiji L1
 - write operacija $WS(x_1)$ koja je dala pročitane vrednosti $R(x_1)$ se pojavljuje i u skupu $WS(x_1; x_2)$ na lokaciji L2 gde isti proces kasnije obavlja upis
 - garantuje da je $W(x_2)$ urađeno na kopiji koja je konzistentna sa kopijom na lokaciji L1
- nema garancija da je $W(x_2)$ urađeno na kopiji koja je konzistentna sa kopijom na lokaciji L1

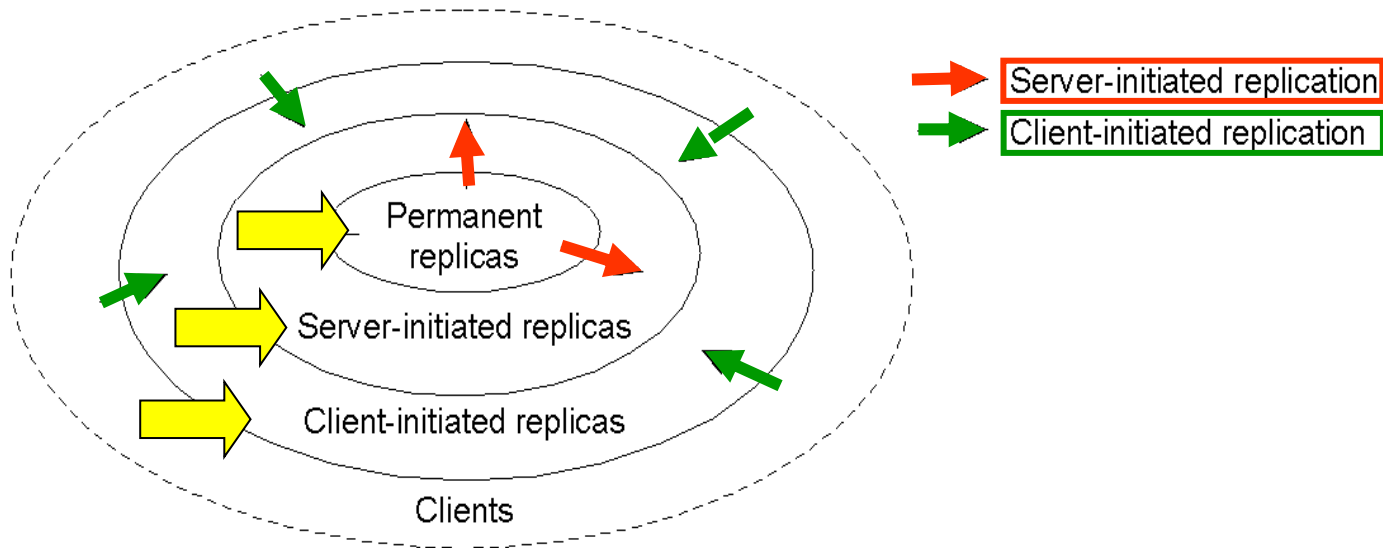
Pozicioniranje replika i protokoli konzistencije

vrste replika

protokoli za ažuriranje replika

Repliciranje sadržaja i pozicioniranje replika

- * Ključno pitanje u DS koji podržava replikaciju je gde, kada i ko može obavljati repliciranje i koji se mehanizmi koriste za održavanje konzistencije.
- * Tri tipa replika:
 - Permanentne replike
 - Replike inicirane od strane servera
 - Replike inicirane od strane klijenta



Permanentne replike

* Mogu se posmatrati kao početni skup replika koje obrazuju distribuirano skladište podataka.

- Broj permanentnih replika je mali.

- Primer: Distribucija Web sajta se javlja u dva oblika

- Repliciranje fajlova koji čine Web sajt na ograničenom broju servera koji se nalaze na jednoj lokaciji (klaster servera). Kada stigne neki zahtev on se prosleđuje jednom od servera, najčešće po kružnom redosledu (round robin)
- Mirroring – Web sajt se kopira na ograničeni broj servera (mirror serveri) koji su geografski dislocirani u Internetu. Klijent odabira jedan od ponuđenih mirror servera.

- Baze podataka takođe mogu biti replicirane na više servera koji zajedno formiraju klaster radnih stanica, ili na veći broj geografski distribuiranih sajtova

Replike inicirane od strane servera

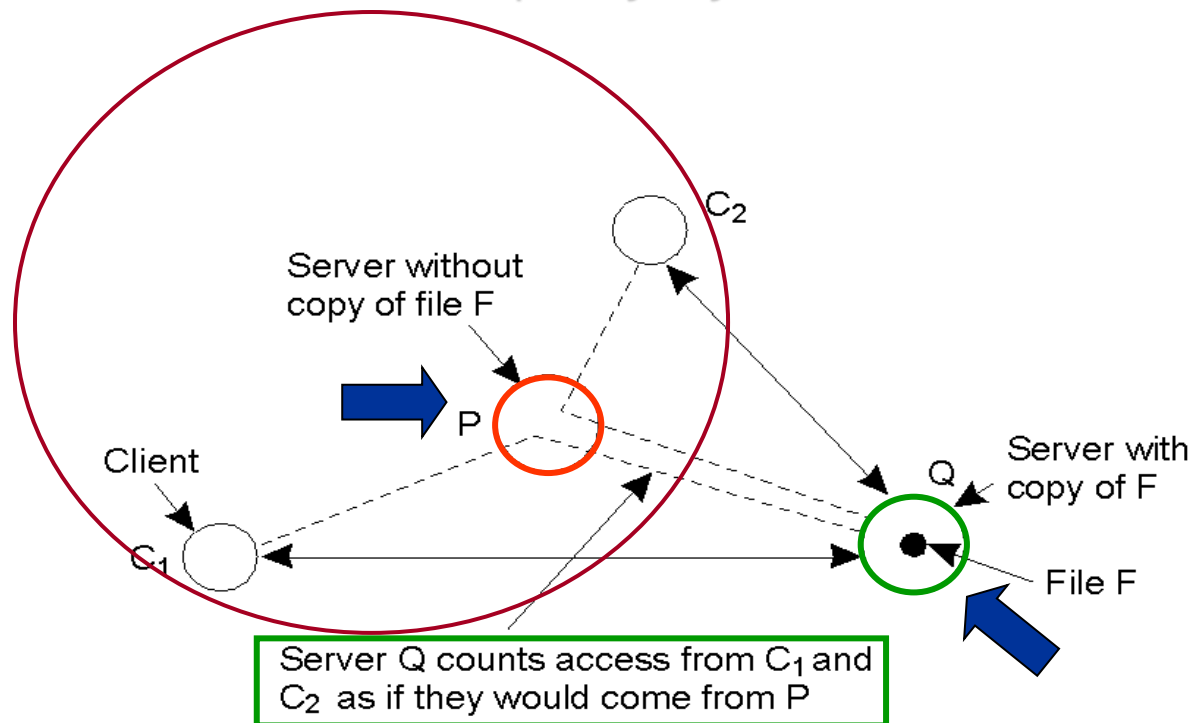
* Formiraju se na inicijativu vlasnika skladišta podataka da bi se poboljšale performanse (vreme odziva) ili rasteretio server ako u nekom trenutku dolazi veliki broj zahteva sa neke udaljene lokacije.

- to su privremene replike postavljene u regione iz kojih dolazi veliki broj zahteva
- Svaki server vodi računa o broju pristupa određenom fajlu i odakle dolaze.
 - Ako je u određenom trenutku broj zahteva sa neke lokacije veliki, svrsishodno je instalirati privremene replike u regionima odakle dolaze zahtevi. (Takve replike zovu se i push replike).
 - Web hosting servis – nudi relativno statičan skup servera u Internetu koji mogu sadržati fajlove i podržati pristup fajlovima koji pripadaju drugom serveru.
 - Da bi se obezbedile optimalne k-ke Web hosting usluge mogu dinamički replicirati fajlove na servere gde su ti fajlovi potrebni da bi se poboljšale performanse, tj. bliže grupi klijenata.

Replike inicirane od strane servera (nast.)

- Gde i kada replike treba kreirati ili obrisati?

- Algoritam dinamičke replikacije uzima dve stvari u obzir: broj obraćanja fajlu i odakle dolaze zahtevi.
- Za svakog klijenta, C , server može odrediti koji je najbliži server u Web hosting servisu.
 - Ako dva klijenta, C_1 i C_2 , imaju isti najbliži server P , svi zahtevi za pristup fajlu F u serveru Q , od strane C_1 i C_2 se jedinstveno registruju u serveru Q : $\text{cnt}_Q(P, F)$, tj. kao da dolaze od servera P .
 - Ako je broj obraćanja fajlu F na serveru Q viši od praga replikacije $\text{rep}(Q, F)$ donosi se odluka da se izvrši replikacija fajla F na server P .



Replike inicirane od strane servera (nast.)

- Kada broj obraćanja fajlu F na serveru P padne ispod praga $\text{del}(P,F)$, fajl može biti uklonjen sa servera P , pod uslovom da to nije posledja (jedina) kopija tog fajla.
 - Na taj način se broj replika redukuje.
 - Vodi se računa da bar jedna replika svakog fajla postoji.
- Ako je broj zahteva za pristup nekom fajlu između praga brisanja i praga replikacije, fajl samo može migrirati sa jednog servera na drugi

- * Serverski inicirane replike se najčešće koriste kao read-only kopije za klijente.
- * Permanentne replike su jedine koje se mogu menjati da bi se sistem održao konzistentnim.

Replike inicirane od strane klijenta

* **Važnu klasu replika čine replike inicirane od strane klijenta.**

- **Poznate su pod nazivom klijent keš.**

- Keš je lokalna memorija koja se koristi od strane klijenta da privremeno zapamti kopiju podataka koje je klijent upravo zahtevao (i dobio).
- Keš može biti lociran na klijent mašini ili na posebnoj mašini u okviru klijentovog LANa.
- Upravljanje kešom je u potpunosti prepušteno klijentu. Skladište podataka iz koga su podaci preuzeti nema nikakvu obavezu u održavanju konzistencije keša.
 - Klijent može uz učešće skladišta utvrditi da li je kopija ustajala ili važeća.
- Klijentski keš se koristi da bi se smanjilo vreme pristupa.
 - Kada klijent želi da pristupi nekim podacima, on se povezuje na najbližu kopiju skladišta podataka, odakle uzima podatke koje želi da pročita ili gde smešta podatke koje je modifikovao.
 - » Ako je u pitanju samo čitanje podataka klijentu se može omogućiti da zapamti podatke u keš.
 - » Sledeći put kada klijent zatraži čitanje istih podataka, može ih preuzeti iz svog lokalnog keša.
 - » Ovo funkcioniše dobro sve dok se replicirani podaci ne promene

Distribucija ažuriranja

* Šta se propagira kod ažuriranja?

● Samo obaveštenje o obavljenom ažuriranu

- Svodi se na invalidaciju ostalih kopija
 - Prednost: mali saobraćaj kroz mrežu
 - Protokoli sa invalidacijom dobro funkcionišu kada je broj ažuriranja mnogo veći od broja čitanja (mnoga ažuriranja mogu biti obavljena lokalno, bez ponovne invalidacije) .

● Podaci koji su ažurirani

- Ažurirani podaci se prenose replikama.
 - Dobro funkcioniše kada je odnos čitanje-upis relativno visok (tj. broj čitanja je mnogo veći od broja upisa).

● Operacija koje su izazvale ažuriranje.

- Ne prenose se podaci, već operacije koje svaka replika treba da obavi (ovakav prilaz poznat je pod nazivom **aktivno repliciranje**)
 - Ažuriranja često mogu biti obavljena uz minimalni saobraćaj
 - Zahteva više procesorskog vremena, naročito ako su operacije kompleksne.

Ko inicira ažuriranje?

* Ažuriranje inicirano od strane servera (push prilaz)

- Ažuriranje se prenosi replikama na inicijativu servera, mada replike to nisu zahtevale.
- Ovaj prilaz se koristi između permanentnih i serverski iniciranih replika, tj. kada je potrebno ostvariti visok nivo konzistencije.

* Ažuriranje na zahtev klijenta (pull prilaz)

- Server ili klijent zahtevaju od drugog servera da mu prosledi ažuriranje ako je postojalo.
 - Npr. uobičajena strategija koja se primenjuje kod Web keševa, je prvo provera da li su keširani podaci još uvek validni.
 - Kada keš primi zahtev za čitanjem podataka koji se još uvek nalaze u kešu, on prvo kontaktira originalni Web server da proverí da li su podaci u međuvremenu bili modifikovani (klijent proziva server da bi ustanovio da li su podaci bili modifikovani).
 - » client to server: {data X, timestamp t_i , OK?}
 - Ako su podaci bili modifikovani, prenose se sa servera u keš, a zatim prosleđuju klijentu
 - » server to client: OK or {data X, timestamp t_i+k }

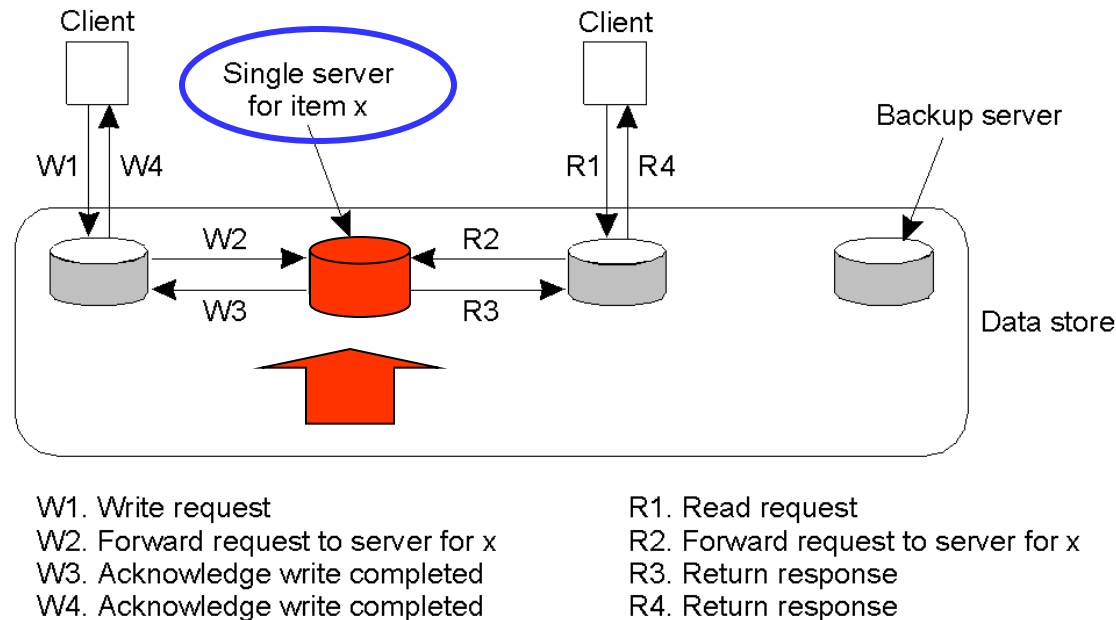
Protokoli konzistencije

- * Protokoli konzistencije opisuju implementaciju određenog modela konzistencije.
- * Tri vrste:
 - Protokoli zasnovani na postojanju primarne kopije (i backup kopija)
 - Write operacija se može izvršiti samo na primarnoj kopiji
 - Protokoli sa više ravnopravnih replika.
 - write operacija može biti inicirana u bilo kojoj replici.
 - Keš koherentni protokoli
 - inicirani od strane klijenta, a ne servera
- * Ako se zahteva postizanje sekvencijalne konzistencije najčešće se koriste protokoli zasnovani na primarnoj kopiji.
 - Kod ovih protokola svaki podatak X u skladištu podataka ima pridruženi primar koji je zadužen za koordinaciju upisa u X.
 - Može se napraviti razlika u odnosu na to da li je primar fiksiran na udaljenom serveru - **Remote-write protokoli** (protokoli sa udaljenim upisom)
 - ili se write operacija može obaviti lokalno nakon što se primar pomeri na proces koji je inicirao write - **Local-write protokoli** (protokoli sa lokalnim upisom)

Remote-write protokoli

* Najjednostavniji protokol baziran na postojanju primarne kopije je protokol kod koga se sve read i write operacije obavljaju na udaljenom serveru.

- U suštini podaci nisu uopšte replicirani, već se nalaze na jednom serveru odakle se ne mogu pomerati.
- Svi zahtevi se upućuju primarnoj kopiji, a ostale kopije služe kao backup

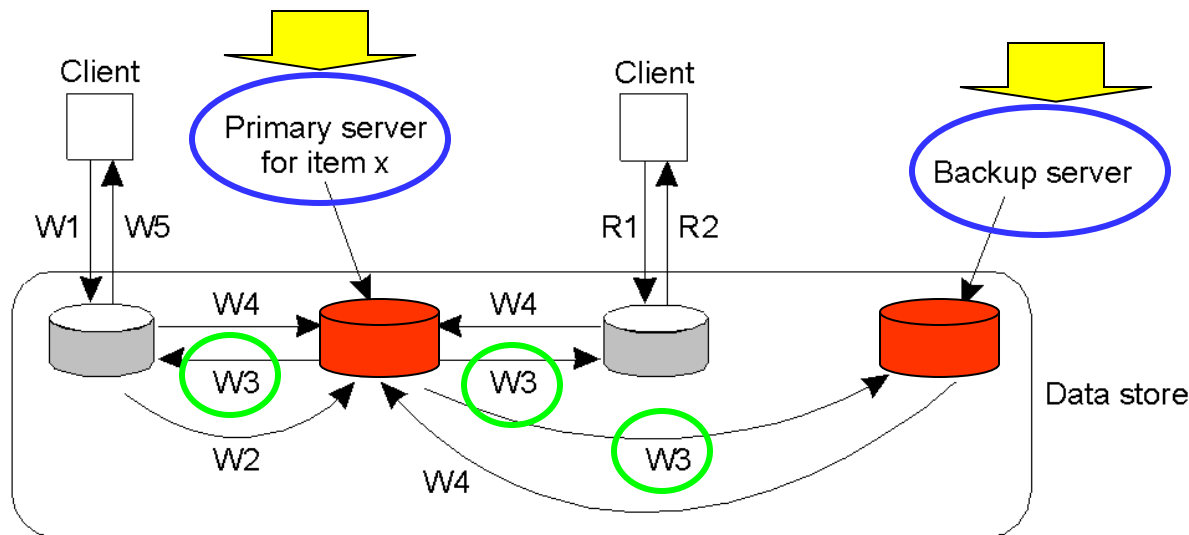


Protokol baziran na postojanju primernog skladišta podataka kod koga se sva čitanja (read) i upisi (write) prosleđuju primaru.

Remote-write protokoli -modifikacija

- * Sa stanovišta konzistencije su mnogo interesantniji protokoli koji dozvoljavaju procesima da obave read na lokalno raspoloživoj kopiji, a operacije write na fiksnoj, primarnoj, kopiji.

- Poznati su pod nazivom primar - backup protokoli



W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

Proces koji čeka da obavi write operaciju nad X, prosleđuje operaciju odgovarajućem serveru (primaru). Primar obavlja ažuriranje na svojoj lokalnoj kopiji, a zatim prosleđuje ažuriranje backup serverima. Svaki backup obavlja ažuriranje i šalje potvrdu primaru. Kada od svih backup servera primi potvrde, primar šalje potvrdu procesu koji je inicirao write operaciju. Može se desiti da proces koji je inicirao write operaciju dugo čeka na dozvolu da nastavi sa radom (ažuriranje je blokirajuća operacija). Druga mogućnost je da se proces blokira samo dok primar ne obavi ažuriranje. Backup serveri se nakon toga ažuriraju.

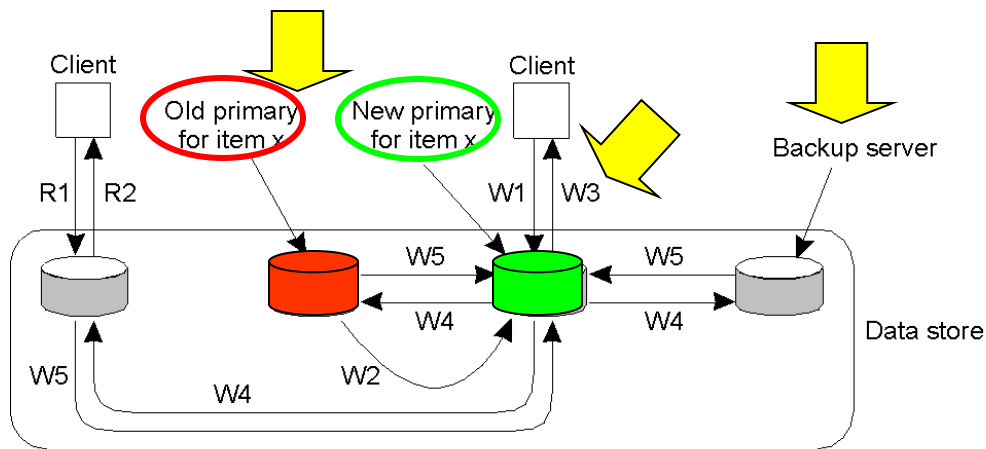
Remote-write protokoli (nast.)

* Protokoli bazirani na postojanju primarne kopije obezbeđuju laku implementaciju sekvencijalne konzistencije.

- Primar može urediti sve dolazne write zahteve na globalno jedinstveni način – svi procesi vide sve write operacije u istom redosledu, bez obzira koji backup server koriste da pročitaju podatke (tj. implementira sekvencijalnu konzistenciju)
- tradicionalno se primenjuje kod distribuiranih baza podataka i fajl sistema koji zahtevaju visoki nivo otpornosti na greške. Replike su najčešće locirane u istom LANu

Local-write protokoli

- * Primarna kopija migrira između procesa koji žele da obave write operaciju.
 - Kada proces želi da ažurira podatak X, on prvo locira primarnu kopiju X, a zatim je privlači u sopstvenu lokaciju.
 - Osnovna prednost ovakvog prilaza je što više uzastopnih write operacija može biti obavljeno lokalno, dok procesi koji obavljaju samo čitanje mogu pristupiti svojim lokalnim kopijama.
 - Ovo se može postići samo ako se koristi neblokirajući protokol koji dozvoljava da se ažuriranja proslede replikama tek kada je primar obavio ažuriranje.



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

Primary-backup protokol u kome se primarna kopija seli između procesa koji žele da izvrše operaciju upisa

Local-write protokoli (nast.)

✱ Ovaj prilaz se koristi u distribuiranim fajl sistemima.

- U ovom slučaju može postojati centralni server preko koga se sve write operacije obavljaju (kao kod remote-write primarnog backup protokola).
- Server privremeno dozvoljava jednoj od replika da obavi seriju lokalnih upisa.
- Kada replika server obavi posao, ažuriranje se prosleđuje centralnom serveru odakle se prosleđuje svim replikama.

Replicirani write protokoli

- * Kod ovih protokola write operacija se može obaviti na više replika, umesto na jednoj kao kod protokola zasnovanim na primarnim kopijama.
 - Aktivna replikacija – operacija se prosleđuje svim replikama
 - Protokoli zasnovani na kvorumu i većinskom glasanju
- * Aktivna replikacija
 - Sve replike su ravnopravne
 - Svaka replika ima proces koji obavlja ažuriranja.
 - Ažuriranja se prosleđuju uz pomoć write operacije (operacija, a ne podaci, se prosleđuje replikama)
 - Problem je što operacije svuda moraju biti obavljene u istom redosledu.
 - Neophodno je implementirati mehanizam potpuno uređene grupne komunikacije.
 - Mogu se iskoristiti Lamportove vremenske markice
 - Alternativno, totalno uređenje se može postići korišćenjem centralnog koordinatora (sekvencer).
 - » Svaka operacija se prvo prosleđuje sekvenceru, koji joj dodeljuje jedinstveni redni broj, a zatim se prosleđuje svim replikama.
 - » Operacije se zatim izvršavaju po redosledu definisanom rednim brojevima (ova implementacija podseća na protokole bazirane na primarnim kopijama)
 - Javlja se problem skaliranja.

Replicirani write protokoli (nast.)

* Protokoli bazirani na kvorumu

- Osnovna ideja je da klijenti zahtevaju i dobijaju dozvolu od više servera pre nego što obave čitanje/upis repliciranih podataka.
- Razmotrimo fajl sistem u kome je fajl repliciran na N servera.
 - Da bi se fajl ažurirao klijent mora kontaktirati najmanje $N/2+1$ (većinu) servera i dobiti dozvolu od njih.
 - Kada se postigne konsenzus, fajl se menja i dodeljuje mu se novi broj verzije koji je isti za sve novonastale fajlove.
 - Da bi pročitao replicirani fajl klijent takođe mora kontaktirati najmanje $N/2+1$ servera i tražiti da mu pošalju broj verzije zajedno sa fajlom.
 - Bar jedan server će sadržati najnoviju verziju fajla.
 - » Npr. ako ima 5 servera od kojih 3 imaju verziju 8, nemoguće je da preostali serveri imaju verziju 9.

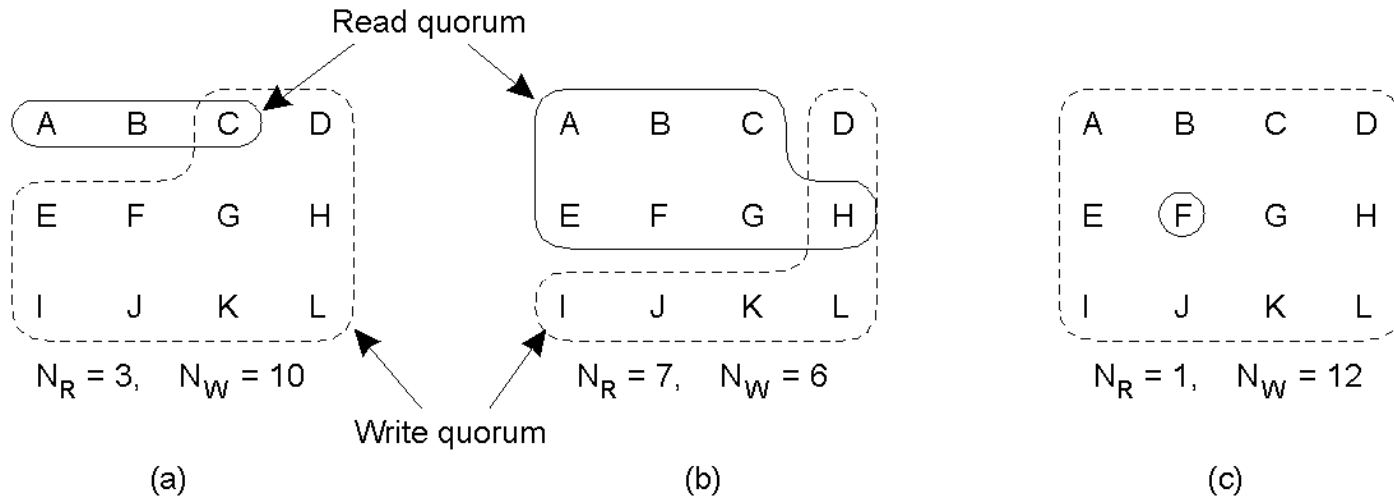
Protokoli bazirani na kvorumu (nast.)

* Opštija šema (Giffordova) kaže

- ako ima N replika, klijent mora da postigne read kvorum na skupu od najmanje N_R ili više servera.
- Da bi modifikovao fajl, write kvorum od najmanje N_W servera se zahteva.
- Moraju da važe sledeći uslovi za N_R i N_W :
 1. $N_R + N_W > N$
 2. $N_W > N / 2$
- Prvi uslov sprečava read-write konflikte, a drugi write-write konflikte (tj. da postoji preklapanje između dva write skupa). Samo ako se odgovarajući broj servera složi, može se obaviti read ili write.

Protokoli bazirani na kvorumu (nast.)

PRIMER



Tri primera algoritma glasanja (fajl je repliciran na N servera): a) korektan izbor read i write skupa, b) izbor koji može dovesti do write-write konflikata i c) korektan izbor, poznat kao ROWA (read one, write all)

Posmatramo sliku a) gde je $N_R=3$ i $N_W=10$. Zamislimo da je najnoviji (poslednji) write kvorum sastavljen od 10 servera od C do L. Svaki od njih uzima novu verziju i broj nove verzije. Svaki sledeći read kvorum od tri servera će morati da sadrži najmanje jedan član iz ovog skupa. Kada klijent vidi brojeve verzije, znaće koji je najskoriji i uzeće taj.

Na slici b) može doći do konflikta zato što je $N_W \leq N/2$. Osobito, ako jedan klijent izabere {A, B, C, E, F, G} kao svoj write skup a drugi klijent izabere {D, H, I, J, K, L} kao svoj write skup, onda dolazi do problema zato što će oba ažuriranja biti prihvaćena bez detekcije da su stvarno u konfliktu.

Situacija na slici c) je interesantna zato što postavlja N_R na 1, čineći mogućim čitanje- read repliciranog fajla nalazeći bilo koju kopiju. Međutim, u ovoj situaciji neophodno je izvršiti ažuriranje svih kopija.